



/-

ClockStandard.lean — A⁵O Clock Standard (A5O-CLK-1.0) revisited in Lean 4.

Purpose: separate the arithmetic content of the January 2026 disclosure ("A⁵O Clock Standard 17.5 ns / 57.142857 MHz", "57 million checks per core") from its physical content, and prove exactly the arithmetic part.

All quantities are integers in fixed units so that no real-number library is needed and every theorem is decidable by the kernel:

time : picoseconds (ps)

frequency : the pair (num, den) meaning num/den hertz

Toolchain: Lean 4.33.1, core library only, no Mathlib, no axioms.

-/

namespace A5O.Clock

/-! ## 1. The clock as stated -/

/-- Stated period: 17.5 ns = 17 500 ps. -/

def periodPs : Nat := 17500

/-- Stated frequency: 400/7 MHz, i.e. $(400 \cdot 10^6) / 7$ Hz. -/

def freqNum : Nat := 400 * 10 ^ 6

def freqDen : Nat := 7

/-- One second in picoseconds. -/

def secondPs : Nat := 10 ^ 12

/-! ## 2. The period and the frequency are the same statement.

$f \cdot T = 1 \Leftrightarrow (400 \cdot 10^6 / 7 \text{ Hz}) \cdot (17\,500 \text{ ps}) = 10^{12} \text{ ps}$

$\Leftrightarrow 400 \cdot 10^6 \cdot 17\,500 = 7 \cdot 10^{12}$ -/

theorem period_matches_frequency : freqNum * periodPs = freqDen * secondPs := by decide

/-! ## 3. Ticks per second is a division, not a measurement.

$10^{12} \text{ ps} / 17\,500 \text{ ps} = 57\,142\,857 \text{ remainder } 2\,500.$ -/

def ticksPerSecond : Nat := secondPs / periodPs

theorem ticksPerSecond_eq : ticksPerSecond = 57142857 := by decide

theorem ticksPerSecond_remainder : secondPs % periodPs = 2500 := by decide

/-! ## 4. "57.142857 MHz" is a six-digit truncation of $400/7$, not an exact value.

$$57\,142\,857 / 10^6 < 400/7 < 57\,142\,858 / 10^6 \quad -/$$

theorem stated_value_is_truncation :

$57142857 * \text{freqDen} < \text{freqNum} * 1 \wedge \text{freqNum} * 1 < 57142858 * \text{freqDen}$:= by decide

theorem stated_value_not_exact : $57142857 * \text{freqDen} \neq \text{freqNum}$:= by decide

/-- The repeating block: $10^6 \equiv 1 \pmod{7}$, so the decimal expansion of $400/7$ repeats with period 6 (the block 142857). -/

theorem decimal_period_six : $10^6 \% 7 = 1$:= by decide

/-! ## 5. Where "57 million checks per core" lives.

A gate that completes k decisions per tick performs $k \cdot \text{ticksPerSecond}$ decisions per second. The number 57 142 857 follows from $k = 1$ and the chosen period. The claim that $k = 1$ holds on any physical core is a HYPOTHESIS about hardware and software; Lean records it, it does not discharge it. -/

/-- A gate characterised only by how many decisions it completes per clock tick. -/

structure Gate where

checksPerTick : Nat

def checksPerSecond (g : Gate) : Nat := g.checksPerTick * ticksPerSecond

/-- The physical claim, stated as a hypothesis on a gate. -/

def OneCheckPerTick (g : Gate) : Prop := g.checksPerTick = 1

/-- Under the hypothesis, the headline number is forced by arithmetic alone. -/

theorem fifty_seven_million (g : Gate) (h : OneCheckPerTick g) :

checksPerSecond g = 57142857 := by

unfold checksPerSecond OneCheckPerTick at *

rw [h, ticksPerSecond_eq]

/-- Without the hypothesis the headline number is not implied by the clock:

a gate completing zero decisions per tick has the same clock and zero throughput. -/

theorem clock_alone_forces_nothing :

```

    ∃ g : Gate, checksPerSecond g = 0 := ⟨⟨0⟩, by decide⟩

```

```

/-- "Checks per core" is "checks per second" only under one further
    hypothesis: one core runs exactly one gate at this clock. That is also
    physical; it is recorded here as a definition, not proved. -/
def checksPerCore (g : Gate) (gatesPerCore : Nat) : Nat :=
    gatesPerCore * checksPerSecond g

```

```

theorem checksPerCore_one (g : Gate) (h : OneCheckPerTick g) :
    checksPerCore g 1 = 57142857 := by
    unfold checksPerCore; rw [fifty_seven_million g h]

```

```

end A5O.Clock

```

```

/-! ## Audit -/
#print axioms A5O.Clock.period_matches_frequency
#print axioms A5O.Clock.ticksPerSecond_eq
#print axioms A5O.Clock.stated_value_is_truncation
#print axioms A5O.Clock.stated_value_not_exact
#print axioms A5O.Clock.fifty_seven_million
#print axioms A5O.Clock.clock_alone_forces_nothing
#print axioms A5O.Clock.checksPerCore_one

```

```

/-!

```

PART II — THE CLOCK AS THE TIME AXIS OF THE PROOF (added 2026-09-19)

The `Nat` that `Hops.lean` quantifies over (`Time := Nat`) and the strict precedence `e.time < a.issued` in `Occurrence.lean` (Theorem 3, "Tick zero") are given a physical unit here: one tick = `periodPs`. Everything below is about ORDER, not speed.

Discipline: every theorem in this file reports zero axioms under `#print axioms`. Core lemmas that carry `propext` (`Nat.mul_mod`, `Nat.add_mul`, `Nat.mul_assoc`, `Nat.sub_*`, `omega`) are deliberately avoided; the few arithmetic facts needed are proved by hand below.

```

-/

```

```

namespace A5O.Clock

```

```

/-- A tick index. The clock makes time discrete and totally ordered. -/

```

abbrev Tick := Nat

/-! ## 6. Strict precedence is at least one tick, and one tick is 17.5 ns. -/

/-- "Strictly before" on a discrete clock means at least one whole tick apart.

This is Occurrence.lean's `e.time < a.issued` with its unit attached. -/

theorem strict_precedence_is_one_tick (a t : Tick) (h : a < t) : a + 1 ≤ t :=
Nat.succ_le_of_lt h

/-- In picoseconds: an appointment at tick a that strictly precedes a decision
at tick t is at least one full period (17 500 ps) earlier. -/

theorem strict_precedence_ps (a t : Tick) (h : a < t) :

a * periodPs + periodPs ≤ t * periodPs := by

have h1 : (a + 1) * periodPs ≤ t * periodPs :=

Nat.mul_le_mul_right _ (Nat.succ_le_of_lt h)

rw [Nat.succ_mul] at h1

exact h1

/-! ## 7. Ledger state as a function of tick; revocation dominance on the clock. -/

/-- A grant is identified by its ledger id. -/

abbrev Grant := Nat

/-- The ledger's revocation record: `RevokedAt g k` means the revocation of
grant g LANDED (was appended) at tick k. Append-only: nothing is ever
removed, so this predicate is all the ledger ever says about revocation. -/

abbrev RevocationLedger := Grant → Tick → Prop

/-- The revocation snapshot as of tick t: the grant stands at t iff no
revocation of it landed at any EARLIER tick. This is `revocation\_as\_of`
given an operational meaning — the state at the start of tick t. -/

def Standing (L : RevocationLedger) (g : Grant) (t : Tick) : Prop :=

∀ k, k < t → ¬ L g k

/-- Revocation dominance on the clock: a revocation landing at tick k defeats
the grant at every strictly later tick. (Hops' `revocation\_cascades`,
stated on the physical time base.) -/

theorem revocation_defeats_later (L : RevocationLedger) (g : Grant) (k t : Tick)

(hrev : L g k) (hlt : k < t) : ¬ Standing L g t :=

fun hs => hs k hlt hrev

/-- Persistence: once a grant does not stand at t, it does not stand at any
later tick. Revocation is not a field that can flip back. -/

theorem not_standing_persists (L : RevocationLedger) (g : Grant) (t t' : Tick)

```

(h : ¬ Standing L g t) (hle : t ≤ t') : ¬ Standing L g t' :=
fun hs' => h (fun k hk => hs' k (Nat.lt_of_lt_of_le hk hle))

```

/-! ## 8. Serialization: one decision per tick means one state per decision. -/

```

/-- A decision record carries the tick it was made at and the tick whose
    snapshot it read (the receipt's `revocation_as_of`). -/
structure Decision where
  tick : Tick
  asOf : Tick

```

```

/-- A gate's schedule under OneCheckPerTick: at most one decision per tick.
    Encoding it as a FUNCTION of the tick is exactly the serialization claim —
    there is no room for two decisions in one moment. -/
abbrev Schedule := Tick → Option Decision

```

```

/-- A well-formed schedule reads the snapshot of its own tick. -/
def ReadsEveryTick (S : Schedule) : Prop :=
  ∀ t d, S t = some d → d.asOf = t

```

```

/-- Under serialization, any two decisions at the same tick are the same
    decision, hence read the same state. There is exactly one appointer of
    record (O_unique) and exactly one state of record per tick. -/
theorem one_state_per_tick (S : Schedule) (t : Tick) (d d' : Decision)
  (h : S t = some d) (h' : S t = some d') : d = d' := by
  rw [h] at h'; exact Option.some.inj h'

```

```

/-- The `revocation_as_of` of a decision is determined by its tick alone. -/
theorem as_of_determined (S : Schedule) (hS : ReadsEveryTick S) (t : Tick)
  (d : Decision) (h : S t = some d) : d.asOf = t :=
  hS t d h

```

/-! ## 9. The exposure window is at most one tick — under ONE hypothesis. -/

```

/-- A gate's verdict on grant g at tick t, as a proposition. -/
abbrev Verdict := Grant → Tick → Prop

```

```

/-- The hypothesis that carries the whole result: the gate PASSES g at t only if
    g stands in the ledger as of t. I.e. the gate reads the ledger every tick.
    This is an operator attestation about a build, the same object-kind as
    NoSideChannel in the deployment record. Lean records it; it does not
    discharge it. -/
def GateReadsLedger (L : RevocationLedger) (pass : Verdict) : Prop :=
  ∀ g t, pass g t → Standing L g t

```

-- Exposure bound: if a gate that reads the ledger every tick passes a grant at tick t, and that grant's revocation landed at tick k, then $t \leq k$.

The only tick at which a revoked grant can still pass is the tick the revocation landed in. The window is at most one tick = 17.5 ns. -/

```
theorem exposure_at_most_one_tick (L : RevocationLedger) (pass : Verdict)
  (hG : GateReadsLedger L pass) (g : Grant) (t k : Tick)
  (hpass : pass g t) (hrev : L g k) : t ≤ k := by
  apply Nat.le_of_not_lt
  intro hlt
  exact revocation_defeats_later L g k t hrev hlt (hG g t hpass)
```

-- Corollary in picoseconds: the passing decision's time never exceeds the revocation's landing time. -/

```
theorem exposure_ps (L : RevocationLedger) (pass : Verdict)
  (hG : GateReadsLedger L pass) (g : Grant) (t k : Tick)
  (hpass : pass g t) (hrev : L g k) : t * periodPs ≤ k * periodPs :=
  Nat.mul_le_mul_right _ (exposure_at_most_one_tick L pass hG g t k hpass hrev)
```

/-! #### Control: a gate that caches for $c \geq 2$ ticks exceeds the window.

Concrete ledger: grant 0 revoked at tick 5. A gate reading the snapshot two ticks stale passes grant 0 at tick 6 — one full tick after the revocation. So the bound is not a property of the clock; it is a property of the hypothesis, and the hypothesis is load-bearing. -/

```
def L₀ : RevocationLedger := fun g k => g = 0 ∧ k = 5
```

```
def stalePass (c : Nat) : Verdict := fun g t => Standing L₀ g (t - c)
```

```
theorem stale_gate_passes_after_revocation : stalePass 2 0 6 := by
  intro k hk ⟨_, hk5⟩
  subst hk5
  exact absurd hk (by decide)
```

```
theorem stale_gate_violates_bound : ¬ (∀ g t k, stalePass 2 g t → L₀ g k → t ≤ k) := by
  intro h
  have := h 0 6 5 stale_gate_passes_after_revocation ⟨rfl, rfl⟩
  exact absurd this (by decide)
```

/-! ## 10. Overlay — recorded at the Chairman's instruction (2026-09-19).

The arithmetic below is proved; its reading is his interpretive overlay and is labelled as such. -/

-- Position of `onebehalf0` in the 13-field canonical receipt body. -/

```
def onebehalfPosition : Nat := 7
```

```
/-- Overlay (a): the clock's denominator is the ONEBEHALFOF position. -/
theorem denominator_is_position_seven : freqDen = onebehalfPosition := rfl
```

```
/-! #### Hand-rolled arithmetic (kept axiom-free on purpose). -/
```

```
/-- Cancel a common +7 (seven applications of successor injectivity). -/
theorem add_seven_cancel (a b : Nat) (h : a + 7 = b + 7) : a = b :=
  Nat.succ.inj (Nat.succ.inj (Nat.succ.inj (Nat.succ.inj
    (Nat.succ.inj (Nat.succ.inj (Nat.succ.inj h))))))
```

```
/-- 7q is never 7q' + r when  $1 \leq r \leq 6$ . -/
theorem seven_mul_ne (r : Nat) (h1 : 1 ≤ r) (h2 : r ≤ 6) :
  ∀ q q' : Nat, 7 * q ≠ 7 * q' + r := by
  intro q
  induction q with
  | zero =>
    intro q' h
    have hr : r ≤ 7 * 0 := by
      have := Nat.le_add_left r (7 * q')
      rw [← h] at this
      exact this
    exact Nat.not_succ_le_zero 0 (Nat.le_trans h1 hr)
  | succ q₀ ih =>
    intro q' h
    cases q' with
    | zero =>
      rw [Nat.mul_succ, Nat.mul_zero, Nat.zero_add] at h
      have h7 : 7 ≤ r := by
        have := Nat.le_add_left 7 (7 * q₀)
        rw [h] at this
        exact this
      exact absurd (Nat.le_trans h7 h2) (by decide)
    | succ q₁ =>
      rw [Nat.mul_succ, Nat.mul_succ, Nat.add_right_comm (7 * q₁) 7 r] at h
      exact ih q₁ (add_seven_cancel _ _ h)
```

```
/--  $c \cdot (7q) = 7 \cdot (cq)$ , by induction, without Nat.mul_assoc. -/
theorem mul_seven_swap (c : Nat) : ∀ q : Nat, c * (7 * q) = 7 * (c * q) := by
  intro q
  induction q with
  | zero => rfl
  | succ q ih =>
```

```

calc c * (7 * (q + 1)) = c * (7 * q + 7) := by rw [Nat.mul_succ]
_ = c * (7 * q) + c * 7 := Nat.mul_add c (7 * q) 7
_ = 7 * (c * q) + 7 * c := by rw [ih, Nat.mul_comm c 7]
_ = 7 * (c * q + c) := (Nat.mul_add 7 (c * q) c).symm
_ = 7 * (c * (q + 1)) := by rw [Nat.mul_succ]

```

/-- Every power of ten is $7q + r$ with $1 \leq r \leq 6$ (the residue never hits 0). -/

```

theorem ten_pow_residue : ∀ n : Nat, ∃ q r : Nat, 1 ≤ r ∧ r ≤ 6 ∧ 10 ^ n = 7 * q + r := by
  intro n
  induction n with
  | zero => exact ⟨0, 1, by decide, by decide, rfl⟩
  | succ n ih =>
    obtain ⟨q, r, h1, h2, hn⟩ := ih
    -- 10^(n+1) = 10 * 10^n = 10 * (7q + r) = 7 * (10q) + 10 r
    have step : 10 ^ (n + 1) = 7 * (10 * q) + 10 * r := by
      rw [Nat.pow_succ, Nat.mul_comm, hn, Nat.mul_add, mul_seven_swap]
    -- 10 r = 7 q'' + r'' by table on r ∈ {1..6}
    have tbl : ∃ q'' r'' : Nat, 1 ≤ r'' ∧ r'' ≤ 6 ∧ 10 * r = 7 * q'' + r'' := by
      match r, h1, h2 with
      | 1, _, _ => exact ⟨1, 3, by decide, by decide, rfl⟩
      | 2, _, _ => exact ⟨2, 6, by decide, by decide, rfl⟩
      | 3, _, _ => exact ⟨4, 2, by decide, by decide, rfl⟩
      | 4, _, _ => exact ⟨5, 5, by decide, by decide, rfl⟩
      | 5, _, _ => exact ⟨7, 1, by decide, by decide, rfl⟩
      | 6, _, _ => exact ⟨8, 4, by decide, by decide, rfl⟩
      | 0, h1, _ => exact absurd h1 (by decide)
      | k + 7, _, h2 => exact absurd (Nat.le_trans (Nat.le_add_left 7 k) h2) (by decide)
    obtain ⟨q'', r'', h1', h2', hr⟩ := tbl
    refine ⟨10 * q + q'', r'', h1', h2', ?_⟩
    rw [step, hr, Nat.mul_add, Nat.add_assoc]

```

/-- Overlay (b): 7 does not divide $400 \cdot 10^n$ for any n — the decimal expansion of $400/7$ never terminates; the remainder is never resolved into the seconds, it is carried forward. -/

```

theorem four_hundred_over_seven_never_terminates :
  ∀ n : Nat, ¬ ∃ q : Nat, 7 * q = 400 * 10 ^ n := by
  intro n ⟨q, hq⟩
  obtain ⟨q', r, h1, h2, hn⟩ := ten_pow_residue n
  -- 400 * 10^n = 7 * (400 q') + 400 r, and 400 r = 7 q'' + r (400 ≡ 1 mod 7)
  have tbl : ∃ q'' : Nat, 400 * r = 7 * q'' + r := by
    match r, h1, h2 with
    | 1, _, _ => exact ⟨57, rfl⟩
    | 2, _, _ => exact ⟨114, rfl⟩
    | 3, _, _ => exact ⟨171, rfl⟩

```

```

| 4, _, _ => exact ⟨228, rfl⟩
| 5, _, _ => exact ⟨285, rfl⟩
| 6, _, _ => exact ⟨342, rfl⟩
| 0, h1, _ => exact absurd h1 (by decide)
| k + 7, _, h2 => exact absurd (Nat.le_trans (Nat.le_add_left 7 k) h2) (by decide)
obtain ⟨q'', hr⟩ := tbl
have : 7 * q = 7 * (400 * q' + q'') + r := by
  rw [hq, hn, Nat.mul_add, mul_seven_swap, hr, Nat.mul_add, Nat.add_assoc]
exact seven_mul_ne r h1 h2 q _ this

/-- Overlay (c), made exact: the tick grid does not land on a second boundary
for seconds 1 through 6, and lands exactly on the seventh. The 2 500 ps
remainder is carried for seven seconds and then resolves. -/
theorem seconds_one_to_six_carry_remainder :
  ∀ s, s < 7 → s ≠ 0 → (s * secondPs) % periodPs ≠ 0 := by decide

theorem seventh_second_resolves : (7 * secondPs) % periodPs = 0 := by decide

/-- Ticks in seven seconds: exactly 400 000 000 — the numerator with its 106
restored, no remainder. -/
theorem ticks_in_seven_seconds : (7 * secondPs) / periodPs = 400000000 := by decide

end A5O.Clock

/-! ## Audit — Part II -/
#print axioms A5O.Clock.strict_precedence_is_one_tick
#print axioms A5O.Clock.strict_precedence_ps
#print axioms A5O.Clock.revocation_defeats_later
#print axioms A5O.Clock.not_standing_persists
#print axioms A5O.Clock.one_state_per_tick
#print axioms A5O.Clock.as_of_determined
#print axioms A5O.Clock.exposure_at_most_one_tick
#print axioms A5O.Clock.exposure_ps
#print axioms A5O.Clock.stale_gate_passes_after_revocation
#print axioms A5O.Clock.stale_gate_violates_bound
#print axioms A5O.Clock.denominator_is_position_seven
#print axioms A5O.Clock.seven_mul_ne
#print axioms A5O.Clock.ten_pow_residue
#print axioms A5O.Clock.four_hundred_over_seven_never_terminates
#print axioms A5O.Clock.seconds_one_to_six_carry_remainder
#print axioms A5O.Clock.seventh_second_resolves
#print axioms A5O.Clock.ticks_in_seven_seconds

```